

C++

Lernzettel & Nachschlagewerk

Kompakte, prüfungs- und praxisorientierte Übersicht des gesamten Stoffs

nach den Vorlesungsfolien von Prof. Dr. Christian Braunagel
DHBW Ravensburg · Abschnitte II–VIII + Lab-Themen

Inhalt dieses Lernzettels

- | | |
|-------------------------------------|---|
| 1. Schnellübersicht (Cheat-Sheet) | 7. Generische Programmierung & STL |
| 2. Von C zu C++ | 8. Exceptions & Smart Pointer |
| 3. OOP-Grundlagen (Klassen) | 9. Code-Design (SOLID, Patterns) |
| 4. UML & Vererbung | 10. Algorithmen & Datenstrukturen |
| 5. UML-Notation ↔ C++ (Spickzettel) | 11. Tooling & Testing (GoogleTest, CMake) |
| 6. Polymorphie | 12. Coding-Conventions |

Legende der Boxen: ■ Definition ■ Merke/Prüfungstipp ■ Syntax ■ Beispiel
■ Best Practice

Inhaltsverzeichnis

1	Schnellübersicht (Cheat-Sheet)	4
2	Von C zu C++ (Abschnitt II)	6
2.1	Dateiendungen	6
2.2	Namespaces	6
2.3	Standard-Ausgabe: <code>std::cout</code> und <code>«</code>	6
2.4	Echter <code>bool</code> -Typ	7
2.5	Strings: <code>std::string</code>	7
2.6	Standard-Eingabe: <code>std::cin</code> und <code>»</code>	7
2.7	Referenzen statt Zeiger	8
2.8	Typsicherheit & Casts	8
2.9	Defaultparameter	8
3	OOP-Grundlagen: Klassen (Abschnitt III)	10
3.1	Klasse vs. struct	10
3.2	Kapselung: Zugriffsspezifizierer	10
3.3	Methoden außerhalb der Klasse + Scope-Operator <code>::</code>	10
3.4	Objekte erzeugen: Stack vs. Heap	10
3.5	Konstruktoren	11
3.5.1	Member-Initialisierungsliste (besserer Stil)	11
3.6	Die Selbstreferenz <code>this</code>	11
3.7	Dynamischer Speicher: <code>new</code> / <code>delete</code>	11
3.8	Destruktoren	12
3.9	Konstante Memberfunktionen	12
3.10	Statische Member	12
4	UML & Vererbung (Abschnitt IV)	14
4.1	Klassendiagramme	14
4.2	Objektdiagramme	14
4.3	Beziehungen zwischen Klassen	14
4.4	Vererbung	15
5	UML-Notation ↔ C++ (Spickzettel)	17
5.1	Aufbau einer Klasse: die drei Felder	17
5.2	Wo kommen Ein- und Ausgabewerte hin?	17
5.3	Attribut-Syntax (Mittelfeld)	18
5.4	Die große Mapping-Tabelle: UML ↔ C++	18
5.5	Komplettbeispiel: UML → C++	18
6	Polymorphie (Abschnitt V)	20
6.1	Binding: statisch vs. dynamisch	20
6.2	Statische Polymorphie (Compile-Zeit)	20
6.3	Dynamische Polymorphie (Laufzeit)	21
6.4	Virtuelle Funktionen & <code>override</code>	21
6.5	Rein virtuelle Funktionen & abstrakte Klassen	22
6.6	Virtueller Destruktor (wichtig!)	22
6.7	Interface vs. abstrakte Klasse	22
7	Generische Programmierung & STL (Abschnitt VI)	23
7.1	Templates	23
7.2	STL – Standard Template Library	24
7.3	for-each-Schleife	24

7.4	Iteratoren	24
7.5	STL-Algorithmen	25
8	Exceptions & Smart Pointer (Abschnitt VII)	26
8.1	Exception Handling: throw / try / catch	26
8.2	Standard-Exceptions	26
8.3	Eigene Exception-Klassen	27
8.4	RAII – Resource Acquisition Is Initialization	27
8.5	Smart Pointer	27
9	Code-Design: SOLID & Patterns (Abschnitt VIII)	29
9.1	SOLID-Prinzipien	29
9.2	Design Patterns	30
10	Algorithmen & Datenstrukturen (Labs)	32
10.1	Sortiervverfahren	32
10.2	Graphen	32
11	Tooling & Testing (Lab 12)	34
11.1	Unit-Tests mit GoogleTest	34
11.2	Build mit CMake	34
12	Coding-Conventions (Kurzreferenz)	36
12.1	Namensgebung	36
12.2	Formatierung	36
12.3	Dateien & Header	36
12.4	Variablen, Typen & Konstanten	36
12.5	Funktionen & Parameterübergabe	36
12.6	Pointer & Referenzen	37
12.7	Kontrollfluss & Ausdrücke	37
12.8	Casts	37
12.9	Klassen & OOP	37
12.10	Speicher & Standardbibliothek	37
12.11	Gefährliche Konstrukte	37

1 Schnellübersicht (Cheat-Sheet)

Eine Seite zum schnellen Nachschlagen beim Programmieren. Details stehen in den jeweiligen Abschnitten.

Programmgrüst & Kompilieren

```
#include <iostream>           // Bibliothek einbinden
int main() {                  // Einstiegspunkt
    std::cout << "Hallo\n";  // Ausgabe
    return 0;                 // 0 = Erfolg
}
```

`g++ -std=c++17 -Wall main.cpp -o main` kompilieren & Warnungen an.

Wichtigste Syntax auf einen Blick

Thema	Kurzform
Ausgabe / Eingabe	<code>std::cout << x; std::cin >> x;</code>
Referenz (Alias)	<code>int& r = a;</code> keine Kopie, nie <code>nullptr</code>
Konstante	<code>const double PI = 3.14159;</code>
Klasse	<code>class A { public: ... private: ... };</code>
Konstruktor (Init-Liste)	<code>A(int x) : x_(x) {}</code>
Vererbung	<code>class B : public A { ... };</code>
Virtuelle Methode	<code>virtual void f(); + void f() override;</code>
Abstrakt / Interface	<code>virtual void f() = 0;</code>
Template	<code>template<typename T> T max(T a, T b);</code>
Container	<code>std::vector<int> v{1,2,3};</code>
Iteration	<code>for (auto& e : v) { ... }</code>
Exception	<code>throw std::runtime_error("..."); / try{}catch(...){}</code>
Smart Pointer	<code>auto p = std::make_unique<A>(...);</code>
Cast	<code>static_cast<int>(x)</code> (nie C-Cast <code>(int)x</code>)

Faustregeln (Best Practices)

- Variablen **sofort initialisieren**, Scope klein halten.
- Member sind **privat**; Zugriff über Methoden.
- Methoden, die nichts ändern, als **const** markieren.
- Große Objekte per **const&** übergeben, kleine per Wert.
- `nullptr` statt `NULL`; `enum class` statt `enum`.
- Überschriebene Methoden mit **override** kennzeichnen.
- Polymorphe Basisklasse braucht **virtuellen Destruktor**.
- Kein manuelles `new/delete` – Smart Pointer / RAII.
- Magische Zahlen durch benannte Konstanten ersetzen.
- Eine Klasse / eine Funktion = **eine Verantwortung**.

Komplexität häufiger Operationen

	vector	map (balanciert)	Bubble/Selection/Insertion
Zugriff per Index	$O(1)$	—	—
Suche	$O(n)$	$O(\log n)$	—
Einfügen am Ende	$O(1)^*$	$O(\log n)$	—
Sortieren (<code>std::sort</code>)	$O(n \log n)$	—	$O(n^2)$

* amortisiert; gelegentliches Reallozieren.

Tipp: Das Inhaltsverzeichnis (oben) ist verlinkt – direkt zum gesuchten Thema springen.

2 Von C zu C++ (Abschnitt II)

C++ ist eine *evolutionäre* Erweiterung von C (Bjarne Stroustrup, "C with Classes"), aber **keine Obermenge**: Nicht jeder C-Code läuft unverändert unter C++. Designziele waren *Zero-Overhead* (keine versteckten Laufzeitkosten, keine Garbage Collection) und *Abstraktion ohne Performanceverlust*.

Merke · Prüfungstipp

Zentrale Neuerungen gegenüber C: **Namespaces**, stärkere **Typsicherheit** (weniger implizite Casts), **Referenzen**, neue Typen `bool` und `std::string`, sowie die Ein-/Ausgabe-Bibliothek `<iostream>` (`std::cout` / `std::cin`).

2.1 Dateiendungen

- Reiner C++-Quellcode: `.cpp`; zugehöriger Header: `.hpp`.
- `.h` nur, wenn der Header auch von C genutzt werden soll (Portabilität).
- Nie den veralteten Header `<iostream.h>` verwenden.

2.2 Namespaces

Ein *Namespace* bündelt Code unter einem Bezeichner und verhindert Namenskonflikte / Shadowing in großen Projekten.

Syntax

`namespace name { ... }` Zugriff: `name::member`

```
namespace mathe { int wert = 10; }
int main() {
    int wert = mathe::wert;    // gleicher Name moeglich
}
```

Best Practice · Konvention

`using namespace std;` spart Schreibarbeit, aber **nur in kleinen Programmen** verwenden – in großen Projekten drohen Namenskonflikte. Besser explizit `std::` schreiben.

2.3 Standard-Ausgabe: `std::cout` und «

Alle Definitionen aus `<iostream>` liegen im Namespace `std`. Der *Einfügeoperator* « schiebt Daten in den Ausgabestrom (von rechts nach links lesbar).

```
std::cout << "Summe: " << a + b << '\n';    // mehrere Werte verketteten
```

Manipulatoren

Manipulatoren formatieren die Ausgabe. Standard-Manipulatoren stecken in `<iostream>`, parametrisierte (z.B. `setw`) in `<iomanip>`.

Manipulator	Wirkung
<code>std::endl</code>	Zeilenumbruch + Puffer leeren (langsamer als <code>'\n'</code>)
<code>std::boolalpha</code>	gibt <code>true/false</code> statt <code>1/0</code> aus
<code>std::fixed</code> / <code>scientific</code>	Festkomma- bzw. Exponentialformat
<code>std::left</code> / <code>right</code>	Bündigkeit (Standard: rechts)
<code>std::hex</code> / <code>dec</code> / <code>oct</code>	Zahlensystem
<code>std::setw(n)</code>	Feldbreite <i>n</i> (nicht sticky!)
<code>std::setprecision(n)</code>	Anzahl Nachkommastellen
<code>std::setfill('c')</code>	Füllzeichen

Merke · Prüfungstipp

Manipulatoren sind *sticky/persistent* – sie gelten bis auf Widerruf. **Einzige Ausnahme:** `setw(n)` wirkt nur auf die nächste Ausgabe. Für Performance `'\n'` statt `std::endl` nutzen.

2.4 Echter bool-Typ

`bool` ist ein eigener fundamentaler Datentyp mit den Werten `true` (intern 1) und `false` (intern 0). Boolesche Ausdrücke liefern `bool`.

```
bool aktiv = true;
bool groesser = (5 > 0); // speichert true
```

2.5 Strings: std::string

In C gab es nur `char`-Arrays mit `'\0'`-Terminator (Buffer-Overflow-Gefahr, feste Größe). `std::string` aus `<string>` wächst dynamisch und ist sicher.

Operation	C++	statt C
Zuweisung	<code>s1 = s2;</code>	<code>strcpy</code>
Verkettung	<code>s1 + s2, s1 += *</code>	<code>strcat</code>
Vergleich	<code>s1 == s2</code>	<code>strcmp</code>
Länge	<code>s1.length()</code>	—
Suche	<code>s1.find("text")</code>	—
Teilstring	<code>s1.substr(start, len)</code>	—
zu C-String	<code>s1.c_str()</code>	—

Best Practice · Konvention

`<string>` immer explizit einbinden, auch wenn es über `<iostream>` manchmal schon verfügbar ist – das ist kein garantiertes Verhalten.

2.6 Standard-Eingabe: std::cin und »

`std::cin` nutzt den *Extraktionsoperator* `»`; die Daten fließen in die Variable. Vorteile gegenüber `scanf`: Typsicherheit (kein `%d`), kein Adressoperator `&`, automatisches Überspringen führender Leerzeichen.

```
int a, b;
std::cin >> a >> b; // Eingaben verkettbar; stoppt bei Leerzeichen/Umbruch
```

Fehlerbehandlung des Streams

Passt der Typ nicht (z.B. `ÄBC` statt `int`), setzt `cin` sein *Fail-Bit*: alle weiteren `cin`-Aufrufe werden übersprungen, die Falscheingabe bleibt im Puffer.

```
#include <limits>
int alter;
if (!std::cin >> alter) { // Fail-Bit prüfen
```

```
std::cout << "Keine Zahl!\n";
std::cin.clear(); // 1. Fehlerzustand zuruecksetzen
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // 2. Puffer leeren
}
```

Merke · Prüfungstipp

Immer das Muster **Rückgabewert prüfen** → **Fehler ausgeben** → **Fehler behandeln** anwenden. `clear()` setzt das Fail-Bit zurück, `ignore(...)` verwirft den restlichen Puffer.

2.7 Referenzen statt Zeiger

Eine *Referenz* ist ein alternativer Name (Alias) für ein bestehendes Objekt.

```
int a = 10;
int& ref = a; // ref ist ein anderer Name fuer a
```

Unterschiede zum Pointer: keine Dereferenzierung nötig, kann (in sauberem Code) nicht `nullptr` sein, und ist nach Bindung **nicht** umbiegbar.

Beispiel: Call-by-Reference vs. Call-by-Pointer (Swap)

```
// C-Stil: Call-by-Pointer // C++-Stil: Call-by-Reference
void swapC(int* a, int* b) { void swapCpp(int& a, int& b) {
    int t = *a; *a = *b; *b = t; int t = a; a = b; b = t;
} }
// Aufruf: swapC(&x, &y); // Aufruf: swapCpp(x, y);
```

Best Practice · Konvention

Primär **Referenzen** nutzen; Zeiger nur dort, wo unausweichlich (z.B. optionale Werte, dynamische Strukturen). Referenzen vermeiden teure Kopien (Call-by-Value).

2.8 Typsicherheit & Casts

C ist *schwach* typisiert (viele implizite Casts erlaubt, z.B. `void* → int*`); C++ ist *stark* typisiert und verlangt explizite Casts.

```
int* p = static_cast<int*>(malloc(sizeof *p)); // C++: expliziter Cast noetig
```

C++-Cast	Zweck
<code>static_cast<T>(x)</code>	normale Umwandlungen (z.B. <code>int → float</code>)
<code>dynamic_cast<T>(x)</code>	sichere Umwandlung in Vererbungshierarchien
<code>const_cast<T>(x)</code>	<code>const</code> -Attribut entfernen
<code>reinterpret_cast<T>(x)</code>	roher Bit-Cast (gefährlich, möglichst meiden)

Merke · Prüfungstipp

Keine C-Casts (int)x mehr! Sie erzwingen jede Umwandlung ungeprüft und sind im Code schwer zu finden. C++-Casts werden zur Compile-Zeit geprüft, sind lesbar und verhindern versehentliches "Wegcasten" von `const`.

2.9 Defaultparameter

Standardwerte werden eingesetzt, wenn beim Aufruf kein Argument übergeben wird. Sie müssen von **rechts nach links** aufgefüllt werden und gehören in die **Deklaration** (Header).


```
void f(int a, int b = 2, int c = 3); // ok
// void f(int a = 1, int b);        // FEHLER: Luecke von rechts
```

3 OOP-Grundlagen: Klassen (Abschnitt III)

In C war die Programmierung *prozedural*: Daten und Logik (Funktionen) sind getrennt. **Objektorientierte Programmierung (OOP)** bündelt beides in *Objekten*. Eine *Klasse* ist die Blaupause (Datentyp), ein *Objekt* ist eine konkrete Instanz (Variable).

Definition: Die vier Säulen der OOP

- **Abstraktion** – nur wesentliche Details zeigen, Komplexität verbergen.
- **Kapselung** – Daten + Methoden in einer Einheit bündeln und Zugriff beschränken.
- **Vererbung** – Eigenschaften/Methoden einer Klasse weitergeben (Wiederverwendung).
- **Polymorphie** – Objekte verschiedener Klassen über einen gemeinsamen Typ behandeln.

3.1 Klasse vs. struct

Klassen werden mit `class` erzeugt und ähneln `structs`. Konvention: Klassenname mit Großbuchstabe.

Merke · Prüfungstipp

Einziger technischer Unterschied: In einer `class` sind Member standardmäßig **private**, in einem `struct` **public**.

3.2 Kapselung: Zugriffsspezifizierer

- **private**: nur innerhalb der eigenen Klasse sichtbar – meist für *Attribute*.
- **public**: überall sichtbar – bildet die *Schnittstelle* (meist Methoden).

```
class Point {
private:
    double x;           // nur intern
    double y;
public:
    void setX(int value); // Schnittstelle
    void setY(int value) { y = value; } // Inline-Definition möglich
};
```

3.3 Methoden außerhalb der Klasse + Scope-Operator ::

Methoden können *inline* in der Klasse oder *außerhalb* definiert werden. Außerhalb verwendet man den *Bereichsauflösungsoperator* `::`.

```
void Point::setX(int value) { // Klasse::Methode
    x = value;
}
```

Anwendungsfälle von `::`: Zugriff auf Namespaces (`std::cout`), Methodendefinition außerhalb der Klasse, Zugriff auf statische Member (`Klasse::member`) und Zugriff auf den globalen Scope (`::number`).

3.4 Objekte erzeugen: Stack vs. Heap

- **Stack** – Zugriff über den Punktoperator `.`:

```
Point p;        p.setX(10);
```

- **Heap** – Zugriff über den Pfeiloperator `->` (nach `new`, siehe unten).

3.5 Konstruktoren

Ein *Konstruktor* wird **automatisch** beim Erzeugen eines Objekts aufgerufen und initialisiert die Attribute.

Merke · Prüfungstipp

Der Konstruktor trägt **denselben Namen wie die Klasse** und hat **keinen Rückgabotyp** (auch nicht `void`).

Art	Beschreibung
Default-Konstruktor	ohne Parameter; existiert automatisch, wenn keiner definiert ist
Parametrisierter K.	setzt Attribute aus Argumenten: <code>Car(int v)</code>
Copy-Konstruktor	erzeugt Kopie eines Objekts: <code>Car(const Car& c)</code>

```
class Car {
public:
    int velocity;
    Car(int v) { velocity = v; }           // parametrisiert
    Car(const Car& c) { velocity = c.velocity; } // Copy (Call-by-Reference!)
};
Car c(120); // ruft parametrisierten Konstruktor
```

3.5.1 Member-Initialisierungsliste (besserer Stil)

Werden Attribute im Konstruktorkörper gesetzt, werden sie erst default-initialisiert und dann überschrieben. Die *Initialisierungsliste* initialisiert direkt.

Syntax

Klasse::Klasse(Param) : member1{param1}, member2{param2} { ... }

```
Plane::Plane(std::string brand, int seats)
: brand{brand}, seats{seats} { } // effizienter; Pflicht bei const-Attributen
```

Merke · Prüfungstipp

Die Initialisierungsliste ist effizienter und **Pflicht bei const- und Referenz-Attributen**. Achtung: Die Member werden in der Reihenfolge der **Klassendeklaration** initialisiert, nicht in der Reihenfolge der Liste.

3.6 Die Selbstreferenz `this`

`this` ist ein *Zeiger* auf das aktuelle Objekt (`ClassName* this`) – deshalb Zugriff mit `->`. Häufigster Einsatz: Namenskonflikt zwischen Parameter und Attribut auflösen.

```
void Plane::setSpeed(int speed) {
    this->speed = speed; // Attribut = Parameter; ohne this-> haette der Parameter Vorrang
}
```

Weitere Anwendungen: Objektadresse übergeben (`f(this)`), Method-Chaining (`return *this;`).

3.7 Dynamischer Speicher: `new` / `delete`

`malloc` reserviert nur *rohen* Speicher und ruft **keinen Konstruktor** auf – bei Klassen mit z. B. `std::string` führt das zu Abstürzen. C++ nutzt daher `new/delete`.

```
int* p1 = new int(42); // Speicher + Initialisierung
int* arr = new int[10]; // Array
MyClass* o = new MyClass(42); // ruft Konstruktor auf -> Zugriff mit o->member

delete p1; // einzelnes Objekt
delete[] arr; // mit new[] erzeugtes Array
```

Merke · Prüfungstipp

Jedes `new` braucht genau ein passendes `delete` (`new[]` → `delete[]`). `delete` ruft erst den **Destruktor** auf und gibt dann den Speicher frei. (Später löst RAII / Smart Pointer dies automatisch – siehe Abschnitt 8.5.)

3.8 Destruktoren

Der *Destruktor* ist das Gegenstück zum Konstruktor und gibt Ressourcen frei.

- Name: `~Klasse()` – kein Rückgabety, keine Parameter, genau einer pro Klasse.
- Wird automatisch aufgerufen bei `delete` oder beim Verlassen des Scopes.

Beispiel: Memory Leak vermeiden

```
class CleanClass {
private:
    int* data;
public:
    CleanClass() { data = new int[1000]; }
    ~CleanClass() { delete[] data; } // ohne diesen Destruktor: Memory Leak!
};
```

Merke · Prüfungstipp

Der vom Compiler erzeugte Standard-Destruktor zerstört nur die Member selbst, gibt aber **keinen mit new reservierten Speicher** frei. Bei dynamischer Speicherverwaltung ist ein eigener Destruktor **zwingend**.

3.9 Konstante Memberfunktionen

Eine `const`-Methode sichert zu, dass sie den Objektzustand nicht ändert (read-only). Das `const` steht **hinter** der Parameterliste.

```
double getSaldo() const { // darf saldo nicht veraendern
    return saldo;
}
```

Best Practice · Konvention

Methoden, die das Objekt nicht verändern, immer als `const` markieren. Bei einem `const`-Objekt (z.B. `const ClassX& x`) sind nur `const`-Methoden aufrufbar.

3.10 Statische Member

`static`-Member gehören der **Klasse als Ganzes**, nicht einem einzelnen Objekt – sie existieren genau einmal.

- **Statische Variablen:** gemeinsamer Speicher für alle Objekte; müssen *außerhalb* der Klasse definiert werden.

- **Statische Methoden:** ohne Objekt aufrufbar (Klasse::methode()); **kein this**, Zugriff nur auf andere statische Member.

```
class Auto {
public:
    static int anzahlAutos;    // Deklaration
    Auto() { anzahlAutos++; }
};
int Auto::anzahlAutos = 0;    // Definition im globalen Scope (Pflicht!)

class Rechner {
public:
    static int add(int a, int b) { return a + b; }
};
int s = Rechner::add(5, 10);  // Aufruf ohne Objekt
```

Typische Anwendungen: Instanzenzähler, objektunabhängige Hilfsfunktionen, Klassenkonstanten, Singleton-Pattern.

4 UML & Vererbung (Abschnitt IV)

UML (Unified Modeling Language, aktuell 2.5.1, ISO/IEC 19505) ist eine grafische Modellierungssprache zur Konstruktion und Dokumentation von Software. Sie ergänzt Flussdiagramme: Während Flussdiagramme *sequentielle Abläufe* zeigen, beantworten Klassendiagramme „*Wer macht was und wie hängt alles zusammen?*“.

4.1 Klassendiagramme

Eine Klasse ist ein Rechteck mit bis zu drei Abschnitten: **Name** / **Attribute** / **Methoden (Operationen)**. Eine zeitliche Abfolge wird nicht dargestellt.

Sichtbarkeit (Visibility)

Symbol	Name	Sichtbar
+	public	überall
-	private	nur in der Klasse
#	protected	in Klasse & abgeleiteten Klassen
~	package	im gesamten Paket

Weitere Notation: Datentyp/Rückgabebetyp nach : ; **statische** Member werden unterstrichen; **abgeleitete** Attribute (berechnet) mit / (z. B. /alter); erweiterte Eigenschaften in { } (z. B. readOnly, ordered). Die Methodeigenschaft {query} entspricht in C++ einer const-Methode.

Syntax: Attribut / Methode in UML

```
sichtbarkeit name : Typ [= Default] {Eigenschaften}
+ berechne(in x : int) : double {query}
```

4.2 Objektdiagramme

Ein Objektdiagramm ist ein *Snapshot* – es zeigt konkrete Instanzen (Objekte) zu einem Zeitpunkt. Unterschiede zum Klassendiagramm: Objektname ist unterstrichen (myCar : Car), Attribute haben nur **konkrete Werte**, Methoden werden **nicht** aufgelistet, keine Sichtbarkeit/Multiplizität. Instanzen von Assoziationen heißen *Links*.

4.3 Beziehungen zwischen Klassen

Neben individuellen Eigenschaften definieren sich Objekte über ihre *Beziehungen*. Die drei Formen unterscheiden sich in der **Stärke der Bindung**.

Assoziation allgemeine Beziehung („kennt“, „verwendet“). Lose gekoppelt, oft über Pointer/Referenzen, **unabhängige** Lebensdauer. UML: einfache Linie, Pfeil zur bekannten Klasse.

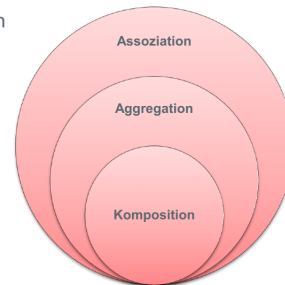
Aggregation „Ganzes-Teile“-Beziehung mit **schwachem** (nicht exklusivem) Besitz; Teile können unabhängig existieren. UML: Linie mit **leerer Raute** auf der Seite des Ganzen.

Komposition „besteht aus“ – **starker, exklusiver** Besitz; Teile existieren nur mit dem Ganzen (gleiche Lebensdauer), meist direkte Member. UML: Linie mit **ausgefüllter Raute** auf der Seite des Ganzen.

```
class Plane { Pilot* pilot; };           // Assoziation: kennt, unabhaengig
class Team { Player* players; };         // Aggregation: hat, unabhaengig
class Plane { Engine engines[4]; };      // Komposition: besteht aus, gekoppelt
```

Aggregation und Komposition

- Aggregation und Komposition sind Unterformen von Assoziationen
- Alle 3 Begriffe beschreiben Beziehungen zwischen Klassen
- Sie unterscheiden sich insbesondere in der Stärke der Bindung zwischen Klassen
- Dabei sind Assoziationen die losesten und Kompositionen die stärksten Verknüpfungen



Prof. Dr. Christian Braunagel

Duale Hochschule Baden-Württemberg Ravensburg

28

Abbildung 1: Assoziation, Aggregation und Komposition im Vergleich (Vorlesungsfolien Abschnitt IV).

Multiplizität

Wie viele Instanzen stehen in Beziehung? Default ist 1.

Notation	Bedeutung
1 / m	genau eins / genau m
* (0..*)	beliebig viele
1..*	mindestens eins
0..1	optional (0 oder 1)
n..m	Bereich (z. B. 2 bis 5)

4.4 Vererbung

Vererbung modelliert eine „**ist-ein**“-**Beziehung**: Eine *Kind-/Unterklasse* (Derived) erbt Attribute und Methoden der *Eltern-/Basisklasse* (Base) und kann sie erweitern oder spezialisieren.

Syntax

```
class Derived : public Base { ... };
```

Merke · Prüfungstipp

Der Zugriffsspezifizierer **protected** ist die Mischform: **protected**-Member sind in der eigenen Klasse **und** in abgeleiteten Klassen sichtbar – aber nicht von außen. **private**-Member sind **nie** in der abgeleiteten Klasse sichtbar.

Vererbungsart (Zugriffsspezifizierer nach dem :)

Steuert, wie geerbte Member in der abgeleiteten Klasse sichtbar werden:

Vererbung	public aus Base	protected aus Base	private aus Base	Bedeutung
: public	bleibt public	bleibt protected	nicht sichtbar	„Derived <i>ist ein</i> Base“
: protected	wird protected	bleibt protected	nicht sichtbar	basiert auf Base (verborgen)
: private	wird private	wird private	nicht sichtbar	nutzt Base intern

public ist der Standard- und Normalfall.

Konstruktoren & Destruktoren bei Vererbung

- Der Konstruktor der **Basisklasse** wird **zuerst** aufgerufen, dann der der abgeleiteten Klasse.
- Hat die Basisklasse **keinen** Default-Konstruktor, muss die abgeleitete Klasse explizit einen passenden Basiskonstruktor aufrufen.
- Destruktoren werden **nicht vererbt** und laufen in **umgekehrter** Reihenfolge (erst Derived, dann Base).

```
class Base {  
    int a;  
public:  
    Base(int a) : a(a) {}  
};  
class Derived : public Base {  
public:  
    Derived(int a) : Base(a) {}    // Basiskonstruktor explizit aufrufen (Pflicht!)  
};
```

Merke · Prüfungstipp

Reihenfolge: **Konstruktion** Base → Derived, **Destruktion** Derived → Base. In UML heißt Vererbung *Generalisierung*: Linie mit **leerer Dreieckspitze**, die stets auf die **Basisklasse** zeigt.

5 UML-Notation ↔ C++ (Spickzettel)

Dieser Abschnitt beantwortet die Frage „*wo schreibe ich was hin?*“: An welche **Position** in einem Klassendiagramm gehören Ein- und Ausgabewerte, und mit welchem Symbol werden **const**, **static**, **virtual** usw. dargestellt – jeweils mit der direkten Übersetzung nach C++.

5.1 Aufbau einer Klasse: die drei Felder

Eine Klasse ist ein Rechteck mit (bis zu) drei Abschnitten. Merksatz: **oben** = **wer**, **Mitte** = **was es hat**, **unten** = **was es kann**.

1. **Name** Klassenname (abstrakte Klasse: *kursiv*).
2. **Attribute** Daten / Zustand des Objekts.
3. **Methoden** Operationen / Verhalten des Objekts.

5.2 Wo kommen Ein- und Ausgabewerte hin?

Das ist der wichtigste Merksatz: **Eingabewerte stehen in den Klammern (...), der Ausgabewert (Rückgabe) steht hinter dem Doppelpunkt nach den Klammern) : Typ**. Die vollständige Operations-Signatur lautet:

Syntax: Methoden-Signatur in UML

```
sichtbarkeit name(richtung param : Typ = default, ...) : Rückgabotyp {Eigenschaft}  
+ einzahlen(in betrag : double) : bool { }
```

Vor jedem Parameter darf eine **Richtung** stehen – genau das ist das „Ein-/Ausgabe“-Konzept:

UML-Richtung	Bedeutung	C++-Umsetzung
in (Default)	reiner Eingabewert , Methode liest nur	Wert: <code>double x</code> ; große Objekte: <code>const T& x</code>
out	reiner Ausgabewert , Methode schreibt zurück	nicht-const Referenz/Zeiger: <code>T& x</code> / <code>T* x</code>
inout	rein und wieder raus (gelesen <i>und</i> geändert)	nicht-const Referenz: <code>T& x</code>
return	der eigentliche Rückgabewert	Rückgabotyp der Funktion

```
// UML: + tausche(inout a : int, inout b : int)  
void tausche(int& a, int& b);  
  
// UML: + teile(in z : int, in n : int, out rest : int) : int  
int teile(int z, int n, int& rest); // Quotient via return, Rest via out-Param  
  
// UML: + getSaldo() : double {query}  
double getSaldo() const; // nur Rueckgabewert, kein Eingabeparameter
```

Merke · Prüfungstipp

Steht keine Richtung da, gilt automatisch **in**. Den Rückgabewert schreibt man fast immer als **) : Typ** ans Ende – **return** als Parameter-Richtung ist zwar erlaubt, aber unüblich. Ein **Defaultwert** kommt mit **= wert** in die Klammer: `+ setVolume(in v : int = 50)` → `void setVolume(int v = 50);`

5.3 Attribut-Syntax (Mittelfeld)

Syntax: Attribut in UML

```
sichtbarkeit /name : Typ [Multiplizität] = Default {Eigenschaft}
- saldo : double = 0.0      + PI : double = 3.14159 {readOnly}
```

Element	C++
Typ : double	double saldo;
Defaultwert = 0.0	double saldo = 0.0;
{readOnly} / {frozen}	const double PI = 3.14159;
/name (abgeleitet/berechnet)	kein eigenes Attribut – berechneter Getter (int getAlter() const)
Multiplizität [0..*]	Container: std::vector<Item> items;

5.4 Die große Mapping-Tabelle: UML ↔ C++

Deine Nachschlage-Liste für const / static / virtual & Co.:

Konzept	UML-Notation	C++
public	+ vor dem Member	public:
private	- vor dem Member	private:
protected	# vor dem Member	protected:
package	~ vor dem Member	(kein echtes Äquivalent)
static (Klassen-Member)	Member <u>unterstrichen</u>	static (vor Typ); Def. außerhalb
const-Methode (ändert Zustand nicht)	{query} hinter der Methode	const <i>hinter</i> der Parameterliste: double getSaldo() const;
const-Attribut	{readOnly} / {frozen}	const <i>vor</i> dem Typ: const int MAX;
abstrakte Methode (rein virtuell)	Methode <i>kursiv</i> oder {abstract}	virtual ... = 0;
abstrakte Klasse	Klassenname <i>kursiv</i> / «abstract»	Klasse mit ≥ 1 rein virtuellen Methode
Interface	Stereotyp «interface»	abstrakte Klasse, nur rein virtuelle Methoden
virtual (überschreibbar, nicht rein)	<i>keine</i> eigene Notation (UML: alles implizit virtuell)	virtual void f();
nicht überschreibbar	{leaf}	final
abgeleitetes Attribut	/name	berechnet im Getter, kein Feld
Defaultwert / Default-Argument	= wert	int v = 50
Konstruktor	Name = Klassenname, evtl. «constructor»	Konto(...), kein Rückgabotyp

Merke · Prüfungstipp

Stolperfalle virtual: Für ein einfaches virtual (überschreibbar, aber *mit* Implementierung) gibt es in UML **kein** eigenes Symbol – UML behandelt alle Methoden ohnehin als virtuell. Markiert (kursiv / = 0) wird nur das **Abstrakte**. Ob eine konkrete Methode in C++ virtual ist, lässt das Diagramm offen. (Details zu virtual/override im Polymorphie-Abschnitt.)

5.5 Komplettbeispiel: UML → C++

Syntax: Klasse Konto in UML

```
- saldo : double = 0.0 (Defaultwert)
anzahlKonten : int (unterstrichen → static)
+ Konto(in name : string) (Konstruktor)
```

```
+ einzahlen(in betrag : double) (in-Parameter)
+ getSaldo() : double {query} (Rückgabe + const)
+ getAnzahl() : int (unterstrichen → static-Methode)
```

```
class Konto {
private:
    double saldo = 0.0;           // = 0.0    -> Defaultwert
    std::string inhaber;
    static int anzahlKonten;      // unterstrichen -> static

public:
    Konto(std::string name)       // Konstruktor: kein Rueckgabety
    : inhaber(name) { anzahlKonten++; }

    void einzahlen(double betrag); // in betrag -> by value

    double getSaldo() const {     // {query} -> const HINTER der Klammer
        return saldo;
    }

    static int getAnzahl() {      // unterstrichen -> static
        return anzahlKonten;     // kein this, nur statische Member
    }
};

int Konto::anzahlKonten = 0;     // static-Definition ausserhalb (Pflicht!)
```

Merke · Prüfungstipp

Kurzformel zum Merken: **Eingabe** → in die Klammern, **Ausgabe (Rückgabe)** → hinter)
∴ const-Methode = {query} und steht *hinter* der Parameterliste; const-Attribut = {readOnly}
und steht *vor* dem Typ. static = unterstrichen, *kursiv* = abstrakt.

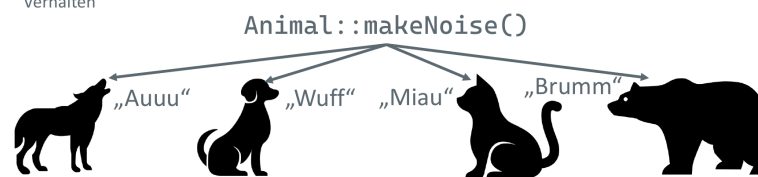
6 Polymorphie (Abschnitt V)

Definition: Polymorphie

Polymorphie bedeutet „**gleicher Aufruf, unterschiedliches Verhalten**“: Die Form eines Bezeichners hängt vom Kontext ab. Man behandelt Objekte verschiedener Klassen über einen gemeinsamen Typ.

Polymorphismus

- Polymorphismus beschreibt das Konzept, die Form eines Bezeichners abhängig vom Kontext zu verändern
- Umgangssprachlich ausgedrückt bedeutet Polymorphismus: gleicher Aufruf, unterschiedliches Verhalten



Prof. Dr. Christian Braunsagel

Duales Hochschule Baden-Württemberg Ravensburg

5

Abbildung 2: „Gleicher Aufruf, unterschiedliches Verhalten“: `Animal::makeNoise()` verhält sich je nach Objekt unterschiedlich (Vorlesungsfolien Abschnitt V).

6.1 Binding: statisch vs. dynamisch

Binding ist die Zuordnung eines Funktionsaufrufs zu einer konkreten Implementierung.

	Static / Early Binding	Dynamic / Late Binding
Entscheidung	zur Compile-Zeit	zur Laufzeit
Funktionsziel	feste Adresse im Code	indirekt über Zeiger (vtable)
ermöglicht	—	Polymorphie

6.2 Statische Polymorphie (Compile-Zeit)

Nutzt Early Binding. Dazu gehören **Funktionsüberladung**, **Operatorüberladung** und **Templates** (Abschnitt VI).

Funktionsüberladung (Overloading)

Mehrere Funktionen mit demselben Namen, aber unterschiedlicher Parameterliste. Der Compiler wählt anhand der Argumente (intern über *Name Mangling*, z. B. `printi`, `printd`).

```
void print(int x)    { std::cout << "int: "    << x << '\n'; }
void print(double x){ std::cout << "double: " << x << '\n'; }
print(5);           // -> print(int)
print(3.14);        // -> print(double)
```

Operatorüberladung

Operatoren (+, *, ==, <, ...) für eigene Typen definieren.

```
class Vec2 {
    double x, y;
public:
    Vec2(double x, double y) : x(x), y(y) {}
    Vec2 operator+(const Vec2& v) const { // a + b -> a.operator+(b)
        return Vec2(x + v.x, y + v.y);
    }
};
```

6.3 Dynamische Polymorphie (Laufzeit)

Bedingungen (alle drei): Vererbung, virtuelle Funktionen, Aufruf über Basisklassen-Zeiger oder -Referenz.

Statischer vs. dynamischer Typ

```
Base* b = new Derived(); // statischer Typ: Base* | dynamischer Typ: Derived
b->foo(); // dynamischer Typ entscheidet -> Derived::foo()
```

Der *statische Typ* (Zeigertyp) ist zur Compile-Zeit bekannt; der *dynamische Typ* (tatsächliches Objekt) steht erst zur Laufzeit fest. *Dynamic Dispatch* wählt anhand des **Objektyps**.

Mechanismus: vtable & vptr

- Jede polymorphe Klasse (mit virtuellen Methoden) hat eine **vtable** – eine Tabelle mit Zeigern auf alle virtuellen Methoden.
- Jedes Objekt besitzt einen versteckten Zeiger **vptr** auf diese Tabelle.
- Beim Aufruf springt das Objekt über den vptr in die vtable, findet die passende Methode und springt an deren Adresse.

6.4 Virtuelle Funktionen & override

Syntax

```
virtual Rueckgabetypp name(Parameter);    in der Basisklasse
Rueckgabetypp name(Parameter) override;   in der abgeleiteten Klasse
```

```
class Tier {
public:
    virtual void lautGeben() { std::cout << "Tier macht ein Laut\n"; }
};
class Hund : public Tier {
public:
    void lautGeben() override { std::cout << "Wuff\n"; }
};
Tier* t = new Hund();
t->lautGeben(); // -> "Wuff" (dynamischer Typ Hund)
```

Merke · Prüfungstipp

Immer override verwenden! Stimmt die Signatur nicht **exakt** mit der virtuellen Funktion überein (gleicher Rückgabetypp, gleiche Parameter, gleiches const), wird ohne override stillschweigend die Basisklassen-Methode aufgerufen – ein schwer findbarer Logik-Bug. Mit **override** prüft der Compiler dies und meldet sofort einen Fehler.

6.5 Rein virtuelle Funktionen & abstrakte Klassen

Eine *rein virtuelle Funktion* hat keine Implementierung (= 0) und muss in jeder abgeleiteten Klasse überschrieben werden. Eine Klasse mit mindestens einer solchen Funktion ist **abstrakt** – von ihr kann **kein Objekt** erzeugt werden.

```
class Shape {                                // abstrakte Klasse / Interface
public:
    virtual double area() const = 0;         // rein virtuell
    virtual ~Shape() = default;             // virtueller Destruktor (s.u.)
};
class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) {}
    double area() const override { return 3.1415 * radius * radius; }
};
// Shape s;                                // FEHLER: abstrakt, nicht instanzierbar
Shape* s = new Circle(2.0);
```

Nutzen: gemeinsame **Schnittstelle** für alle Kindklassen, echte Polymorphie und Design-Sicherheit (fehlende Überschreibung verhindert Instanziierung).

6.6 Virtueller Destruktor (wichtig!)

Merke · Prüfungstipp

Wird eine Klasse polymorph genutzt (Löschen über Basisklassen-Zeiger), **muss** sie einen **virtual-Destruktor** haben. Ohne ihn wird nur der Basis-Destruktor aufgerufen $\Rightarrow$ der Destruktor der abgeleiteten Klasse läuft nicht $\Rightarrow$ **Speicherleck / undefiniertes Verhalten**.

```
class Shape { public: virtual ~Shape() { } }; // virtual -> korrekt
class Circle : public Shape { public: ~Circle() { /* gibt Ressourcen frei */ } };

Shape* s = new Circle();
delete s; // ruft ~Circle() UND ~Shape() in richtiger Reihenfolge
```

6.7 Interface vs. abstrakte Klasse

In C++ werden beide als abstrakte Klasse umgesetzt; ein *Interface* ist die Sonderform mit **ausschließlich** rein virtuellen Methoden.

Abstrakte Klasse	Interface
gemeinsame Basis + Verhalten	reines Verhalten (Vertrag)
echte Attribute möglich	nur Methodensignaturen
Semantik „ist eine Art von“	Semantik „kann etwas“
darf konkrete Methoden haben	nur rein virtuelle Methoden

UML: Abstraktes wird *kursiv* und/oder mit Stereotyp **«abstract»** bzw. **«interface»** gekennzeichnet; Implementierung via gestricheltem Pfeil oder Lollipop-Symbol. In UML ist jede Methode implizit virtuell.

7 Generische Programmierung & STL (Abschnitt VI)

Definition: Generische Programmierung

„Schreibe Code einmal – nutze ihn für viele Typen.“ Algorithmen und Datenstrukturen werden **unabhängig vom konkreten Datentyp** spezifiziert (statt `max_int`, `max_double`, ... nur ein `max<T>`).

Vorteile: **Wiederverwendung** (DRY), **Typsicherheit** (Prüfung zur Compile-Zeit), **Effizienz** (eigene Codeversion je Typ = *Monomorphismus*, kein Laufzeit-Overhead), **Flexibilität** (Anforderungen an Eigenschaften statt feste Typen).

7.1 Templates

Templates sind der zentrale Mechanismus: Code mit Platzhaltern für Typen/Werte, der **zur Compile-Zeit** in konkreten Code übersetzt wird.

Funktions-Template

```
template <typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
max(3, 5);           // -> max<int>
max(2.5, 1.5);       // -> max<double>
// max(3, 5.5);      // FEHLER: T uneindeutig (int vs double)
```

`template <...>` steht direkt vor der Funktion und darf nicht abgetrennt werden. Funktioniert nur, wenn die verwendeten Operatoren für T existieren.

Klassen-Template

```
template <typename T>
class Box {
    T value;
public:
    Box(T v) : value(v) {}
    T get() { return value; }
};
Box<int> b1(10);      // Typ explizit (noetig vor C++17)
Box b2(3.14);        // ab C++17 deduzierbar
```

Mehrere Typ- und Nicht-Typ-Parameter

```
template <typename T, typename U>           // mehrere Typparameter
void printPair(T a, U b) { std::cout << a << ", " << b << '\n'; }

template <typename T, int N>                // Nicht-Typ-Parameter (Wert!)
class Array { public: T data[N]; };
Array<int, 10> a;                          // T=int, N=10 (N muss zur Compile-Zeit bekannt sein)
```

Template-Spezialisierung

Spezielle Implementierung für einen konkreten Typ – hat **Priorität** vor dem allgemeinen Template.

```
template <typename T> T process(T value) { return value + value; }

template <>                                     // Spezialisierung fuer bool
```

```
bool process<bool>(bool value) { return !value; }
```

Merke · Prüfungstipp

Templates werden erst **beim Verwenden** instanziiert und für jede Übersetzungseinheit separat erzeugt. Deshalb müssen Templates **inkl. ihrer Logik in der Header-Datei (.hpp)** stehen – Implementierung in einer .cpp führt zu *Undefined Behaviour*.

7.2 STL – Standard Template Library

Die STL kombiniert **Container** (Objektsammlungen), **Iteratoren** (einheitlicher Durchlauf) und **Algorithmen** (Such-/Sortierfunktionen). Vorteile: optimierte Performance, weniger Speicherfehler, standardisierte Interfaces.

Container-Kategorien

Kategorie	Beschreibung / Beispiele
Sequence	geordnet, Zugriff per Index/Iterator (<code>vector</code> , <code>list</code> , <code>array</code>)
Adapter	Wrapper, der einen Container einschränkt (<code>stack</code> , <code>queue</code>)
Associative	automatisch sortiert, Zugriff per Key, Baum (<code>map</code> , <code>set</code>)
Unordered	Key-basiert, unsortiert, Hash (<code>unordered_map</code>)

Die wichtigsten Container

	<code>std::vector</code>	<code>std::map</code>
Modell	dynamisches Array	Key→Value (sortiert)
Speicher	zusammenhängend	Baumstruktur
Indezzugriff	$v[i] - O(1)$	—
Suche	$O(n)$	per Key $O(\log n)$
typisch	<code>push_back</code> , <code>front</code> , <code>back</code>	<code>m["k"]=v</code> , <code>insert</code> , <code>erase</code>

`std::list` ist eine **doppelt verkettete Liste**: kein Indezzugriff, aber schnelles Einfügen/Löschen an beliebiger Stelle (`push_front`, `push_back`).

```
std::vector<int> v = {1, 2, 3};
v.push_back(4);           // anhaengen
std::cout << v[1] << v.front() << v.back();

std::map<std::string, int> m;
m["Alice"] = 25;          // anlegen oder aendern
m.insert({"Bob", 30});
m.erase("Alice");
```

Gemeinsame Konzepte aller Container: `c.size()`, `c.empty()`, `c.insert(...)`, `c.erase(...)`, `for-each`-Schleife und STL-Algorithmen.

7.3 for-each-Schleife

```
std::vector<int> v = {1, 2, 3};
for (int x : v)        { std::cout << x << " "; } // Kopie
for (auto& x : v)      { x *= 2; }                // Referenz: veraendert v
```

Durchläuft alle Elemente; kein Indezzugriff, Reihenfolge/Richtung nicht anpassbar.

7.4 Iteratoren

Ein *Iterator* ist ein **verallgemeinerter Zeiger**: zeigt auf ein Element, kann weiterbewegt werden und erlaubt Zugriff. Die `for-each`-Schleife nutzt intern Iteratoren.


```
for (std::vector<int>::iterator it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << " ";    // *it: Zugriff, ++it: naechstes Element
}
```

`begin()` liefert den ersten, `end()` zeigt **hinter** das letzte Element – der Bereich ist halboffen [`begin`, `end`).

Iterator-Kategorien (hierarchisch, je außen stärker)

Kategorie	Kann	Operationen	Container
Input / Output	einmal lesen <i>bzw.</i> schreiben	<code>*i, ++</code>	Streams
Forward	mehrfach lesen+schreiben	<code>*i, ++</code>	<code>forward_list</code>
Bidirectional	vorwärts & rückwärts	<code>++, -</code>	<code>list, map</code>
Random-Access	beliebiger Index	<code>++ - += [] < ></code>	<code>vector</code>

Merke · Prüfungstipp

Beim Iterator: `.` greift auf Member des *Iterators* zu, `->` auf Member des *Elements*. (`it->name` statt `(*it).name`.)

7.5 STL-Algorithmen

Generische Funktionen, die auf **Bereichen** [`begin`, `end`] arbeiten – *nicht* auf Containern. Dadurch containerunabhängig.

Syntax

```
std::algorithmus(begin, end, ...);
```

Algorithmus	Wirkung
<code>std::sort</code>	sortiert aufsteigend ($O(n \log n)$)
<code>std::find</code>	erstes passendes Element
<code>std::count</code>	Anzahl eines Werts
<code>std::accumulate</code>	Summe (aus <code><numeric></code>)
<code>std::copy / transform</code>	kopieren / Funktion anwenden
<code>std::reverse / replace</code>	umkehren / Werte ersetzen

```
#include <algorithm>
std::vector<int> v = {4, 1, 7, 3};
std::sort(v.begin(), v.end());           // 1 3 4 7
auto it = std::find(v.begin(), v.end(), 3);
if (it != v.end()) std::cout << "gefunden: " << *it;
```

Best Practice · Konvention

Trade-offs: Templates erzeugen oft sehr lange, schwer lesbare Compiler-Fehler, höhere Codekomplexität sowie längere Compile-Zeiten und größere Binärdateien (*Code-Bloat*), da pro Typ eigener Code entsteht.

8 Exceptions & Smart Pointer (Abschnitt VII)

In C musste nach fast jedem Aufruf der Rückgabewert geprüft werden – Normal- und Fehlerfall vermischen sich, Kontext fehlt. C++ trennt beides mit **Exceptions**.

8.1 Exception Handling: throw / try / catch

Definition: Exception

Eine Exception ist ein **Objekt**, das eine Fehlersituation repräsentiert und den normalen Kontrollfluss unterbricht, um an geeigneter Stelle behandelt zu werden.

Ablauf in drei Schritten:

- `try { ... }` – markiert einen Block, in dem ein Fehler auftreten kann.
- `throw obj;` – wirft eine Exception.
- `catch (Typ e) { ... }` – fängt sie und behandelt sie.

```
int divide(int a, int b) {
    if (b == 0) throw std::invalid_argument("Division durch 0");
    return a / b;
}
int main() {
    try {
        std::cout << divide(10, 0) << '\n';
    } catch (const std::exception& error) { // const Referenz!
        std::cout << "Fehler: " << error.what();
    }
    return 0; // Programm laeuft nach dem catch normal weiter
}
```

Merke · Prüfungstipp

Wird eine Exception nicht lokal behandelt, wandert sie den Aufrufstack nach oben (*Stack Unwinding*); dabei werden lokale Objekte korrekt über ihre Destruktoren zerstört. Wird sie **gar nicht** gefangen, beendet `std::terminate()` das Programm. Auf jeden `try`-Block muss direkt ein `catch`-Block folgen.

Mehrere catch-Blöcke

Werden von oben nach unten geprüft; der erste passende wird ausgeführt, der Rest übersprungen. Ein allgemeiner Auffangblock steht **immer am Ende**.

```
try { /* ... */ }
catch (const std::invalid_argument& e) { /* spezifisch */ }
catch (const std::exception& e) { /* allgemeiner, bevorzugt */ }
catch (...) { /* letzte Sicherheitsstufe */ }
```

8.2 Standard-Exceptions

Einfache Typen (`throw 42;`) sind ungeeignet: keine Aussagekraft, keine Zusatzinfos, schlechte Unterscheidung. Besser: Klassen aus `<stdexcept>`, die von `std::exception` erben und die virtuelle Methode `what()` besitzen.

Kategorie	Beispiele
Logische Fehler / ungültige Eingaben	<code>invalid_argument</code> , <code>length_error</code> , <code>logic_error</code>
Laufzeitfehler	<code>runtime_error</code> , <code>overflow_error</code> , <code>out_of_range</code>
Speicher- & Typfehler	<code>bad_alloc</code> , <code>bad_cast</code>

8.3 Eigene Exception-Klassen

Für fachlich spezifische Fehler – erben von einer passenden Standard-Exception.

```
class DivByZeroErr : public std::logic_error {
public:
    DivByZeroErr() : std::logic_error("Division durch 0") {}
};
// throw DivByZeroErr();
// catch (const DivByZeroErr& error) { std::cout << error.what(); }
```

Best Practice · Konvention: Leitlinien

- Standard-Exceptions bevorzugen; eigene Klassen nur bei fachlich relevanten Fehlern.
- **Be specific** – die spezifischste sinnvolle Exception werfen.
- try-Blöcke klein halten; catch dort, wo man den Fehler versteht.
- Exceptions **als const Referenz** fangen (verhindert *Object Slicing*).
- catch (const std::exception& e) ist besser als catch (...).

8.4 RAI – Resource Acquisition Is Initialization

Definition: RAI

Eine Ressource (Speicher, Datei, DB-Verbindung, ...) wird beim **Erzeugen** eines Objekts erworben und beim **Zerstören** automatisch freigegeben – Ressourcen sind an die Objektlebensdauer gebunden. Umgesetzt über Konstruktor + Destruktor.

```
class File {
    FILE* f;
public:
    File(const std::string& name) { f = fopen(name.c_str(), "r"); }
    ~File() { if (f) fclose(f); } // wird IMMER aufgerufen, auch bei Exception
};
void foo() {
    File f("test.txt");
    if (error) return; // Datei wird trotzdem sauber geschlossen
}
```

Vorteile: keine Memory Leaks, Exception-Sicherheit, klarer Besitz, bessere Wartbarkeit. Voraussetzung: Objekt ist **lokal** (kein roher new-Pointer ohne Lebensdauerverwaltung).

8.5 Smart Pointer

Rohe Pointer verursachen Memory Leaks, Dangling Pointer, Double-Free und unklare Besitzverhältnisse. *Smart Pointer* (Wrapper-Klassen aus <memory>) verhalten sich wie Pointer (überladen * und ->), übernehmen aber die Speicherverwaltung automatisch (RAII) und rufen selbständig delete auf.

	Besitz	#Besitzer	Zähler	Zerstörung
unique_ptr	exklusiv	genau 1	nein	beim Verlassen des Besitzers
shared_ptr	geteilt	beliebig	ja	wenn letzter shared_ptr weg
weak_ptr	kein Besitz	0	nutzt, erhöht nicht	nie

std::unique_ptr – exklusiver Besitz

```
auto p1 = std::make_unique<int>(5); // reserviert intern mit new
// auto p2 = p1; // FEHLER: Kopieren verboten
auto p2 = std::move(p1); // Besitz uebertragen -> p1 == nullptr
```

Kein manuelles `delete` mehr (keine Double-Delete-Gefahr). Verwendung ohne Besitzübergabe: `print(*p)` (Referenz) oder `foo(p.get())` (roher Pointer, Besitz bleibt bei `p`). Ideal für Kompositionen und überall, wo früher `new` stand – so effizient wie ein roher Pointer.

`std::shared_ptr` – geteilter Besitz

```
auto p1 = std::make_shared<int>(5);  
auto p2 = p1;           // beide besitzen; Zaehler = 2  
std::cout << p1.use_count(); // Referenzzahler abfragen
```

Das Objekt wird erst gelöscht, wenn der **letzte** Besitzer verschwindet (*Referenzzähler*). Langsamer als ein roher Pointer wegen des Zählers.

`std::weak_ptr` – löst das Zyklusproblem

Besitzen sich zwei `shared_ptr` gegenseitig ($A \rightarrow B, B \rightarrow A$), bleibt der Zähler stets > 0 – die Destruktoren laufen nie (Memory Leak). Ein `weak_ptr` zeigt auf ein von `shared_ptr` verwaltetes Objekt, **erhöht den Zähler aber nicht**.

```
std::shared_ptr<A> temp = objB->a.lock(); // weak_ptr nicht direkt dereferenzierbar  
if (temp) { /* Objekt existiert noch */ } // sonst temp == nullptr
```

Merke · Prüfungstipp

Faustregel: **Wann immer eine Klasse/Funktion eine Ressource besitzt, Smart Pointer verwenden.** `unique_ptr` als Standard (exklusiv), `shared_ptr` nur bei echtem geteiltem Besitz, `weak_ptr` zum Aufbrechen von Zyklen.

9 Code-Design: SOLID & Patterns (Abschnitt VIII)

Software Engineering ist die systematische Entwicklung, Planung, Implementierung, Testung und Wartung von Software. Gutes Design reduziert **Kopplung** und fördert **klare Verantwortlichkeiten**.

9.1 SOLID-Prinzipien

Fünf Designprinzipien für skalierbare, erweiterbare und flexible (OO-)Software.

S – Single Responsibility (SRP)

Eine Klasse sollte **genau einen Grund zur Änderung** haben. Mehrere Aufgaben in einer Klasse ⇒ Änderungen beeinflussen sich gegenseitig, schlechtere Test- und Wiederverwendbarkeit.

```
// Verletzung: drei Verantwortlichkeiten
class Report {
    void generate();
    Report&); };
    void saveToFile();
    ; };
    void print();
};

// Korrektur: aufteilen
class Report    { void generate(); };
class ReportSaver { void saveToFile(const
class ReportPrinter{ void print(const Report&
```

Merke · Prüfungstipp

„Eine Aufgabe“ heißt **nicht** „eine Methode“ – eine Klasse darf viele Methoden haben, solange sie zur gleichen Verantwortung gehören.

O – Open/Closed (OCP)

Klassen sollen **offen für Erweiterung, aber geschlossen für Änderung** sein: neuen Code hinzufügen statt funktionierenden Code umschreiben (vermeidet Bugs/Regressionen). Umsetzung über Interfaces, abstrakte Klassen, virtuelle Methoden, Polymorphie.

```
// statt if (shape=="circle") ... else if ... -> Vererbung + override
class Shape { public: virtual double area() const = 0; virtual ~Shape() = default; };
class Circle : public Shape { public: double area() const override { return 10.0; } };
```

L – Liskov Substitution (LSP)

Objekte einer abgeleiteten Klasse müssen sich **wie** die der Basisklasse verhalten – sie müssen überall austauschbar sein. Drei Regeln:

1. Vorbedingungen dürfen **nicht verschärft** werden (Derived verlangt nicht mehr als Base).
2. Nachbedingungen dürfen **nicht abgeschwächt** werden (Derived garantiert mind. so viel wie Base).
3. Invarianten müssen **erhalten** bleiben (global gültige Zustände nicht verletzen).

I – Interface Segregation (ISP)

Klassen sollen nicht von Methoden abhängen, die sie nicht brauchen. Lieber **mehrere kleine, spezialisierte Interfaces** als ein großes. (ISP ist das SRP für Interfaces.)

```
// Verletzung: BasicPrinter muss scan() implementieren, wirft aber nur Exception
class Machine { public: virtual void print() = 0; virtual void scan() = 0; };
// Besser: getrennte Interfaces Printer und Scanner
```

D – Dependency Inversion (DIP)

High-Level-Module sollen nicht von konkreten Implementierungen abhängen, sondern von **Abstraktionen** (Interfaces). Entkoppelt Fachlogik von technischen Details (DB, API, Framework).

```
class Database { public: virtual void save_order() = 0; }; // Abstraktion
class MySQLDatabase : public Database { public: void save_order() override {} };
class OrderService {
    Database& db; // kennt nur die Abstraktion
public:
    OrderService(Database& db) : db(db) {}
};
```

9.2 Design Patterns

Definition: Design Pattern

Bewährte, wiederverwendbare Lösungsbaupläne für wiederkehrende Design-Probleme – kein konkreter Code, sondern Beschreibung von Problem, Lösungsansatz und beteiligten Komponenten. Standardwerk: *Gang of Four* (Gamma et al., 1994).

Kategorie	Zweck	Beispiele
Creational (Erzeugung)	Objekterstellung regeln/entkoppeln	Factory, Singleton, Builder
Structural (Struktur)	Klassenaufbau vereinfachen	Adapter, Decorator, Proxy
Behavioral (Verhalten)	Kommunikation/Verantwortung	Observer, Strategy, Command, State

Factory Method (Creational)

Objekte werden nicht überall direkt mit `new` erzeugt, sondern zentral über eine *Factory* – gekapselt, austauschbar, erweiterbar, testbar.

```
class SensorFactory {
public:
    static std::unique_ptr<Sensor> create_sensor(const std::string& type) {
        if (type == "temperature") return std::make_unique<TemperatureSensor>();
        if (type == "pressure") return std::make_unique<PressureSensor>();
        return nullptr;
    }
};

auto sensor = SensorFactory::create_sensor("temperature");
sensor->read();
```

Vorteil: `main` kennt nur noch die Factory + die abstrakte Basisklasse `Sensor` (DIP); neue Sensoren erfordern nur eine Änderung der Factory (OCP).

Observer (Behavioral)

Ein *Subject* informiert automatisch registrierte *Observer*, wenn sich sein Zustand ändert. Statt dass z. B. ein `Sensor` `display.update()`, `logger.update()`, ... direkt aufruft, registrieren sich die Observer über ein gemeinsames Interface $\Rightarrow$ lose Kopplung, leicht erweiterbar (OCP).

Adapter (Structural)

Übersetzt zwischen zwei inkompatiblen Schnittstellen: Eine fremde Klasse (z. B. `LegacyTemperatureDevice`) soll genutzt werden, ohne sie zu ändern. Der Adapter sieht nach außen wie das erwartete `Sensor`-Interface aus, nutzt intern aber die Fremdklasse. Ideal zum Integrieren von Legacy-/Herstellerbibliotheken.

Best Practice · Konvention

SOLID + Patterns zielen auf dasselbe: **geringe Kopplung, hohe Kohäsion**. Patterns schaffen außerdem eine gemeinsame Sprache – „das ist ein Strategy Pattern“ ersetzt lange Erklärungen.

10 Algorithmen & Datenstrukturen (Labs)

Aus den Labs 7 und 9: einfache Sortierverfahren und Graphen. Ergänzt die STL-Algorithmen aus dem Abschnitt *Generische Programmierung & STL*.

10.1 Sortierverfahren

Beide Verfahren sortieren **in-place** und haben die Zeitkomplexität $O(n^2)$. Sie eignen sich zum Verstehen der Grundidee – in der Praxis nutzt man `std::sort` ($O(n \log n)$).

Insertion Sort (Einfügen)

Jedes Element wird an die richtige Stelle des bereits sortierten linken Teils „einsortiert“. Best Case $O(n)$ (fast sortiert), **stabil**.

```
for (std::size_t i = 1; i < values.size(); ++i) {
    int key = values[i];
    std::size_t j = i;
    while (j > 0 && values[j - 1] > key) {    // groessere Elemente nach rechts schieben
        values[j] = values[j - 1];
        --j;
    }
    values[j] = key;                        // key an richtiger Stelle einfüegen
}
```

Selection Sort (Auswahl)

Sucht in jedem Durchlauf das Minimum des Rests und tauscht es nach vorne. Immer $O(n^2)$ (auch im Best Case), **nicht stabil**.

```
for (std::size_t i = 0; i + 1 < values.size(); ++i) {
    std::size_t min_index = i;
    for (std::size_t j = i + 1; j < values.size(); ++j)
        if (values[j] < values[min_index]) min_index = j;
    if (min_index != i)
        std::swap(values[i], values[min_index]);    // Minimum nach vorne
}
```

Verfahren	Best	Average/Worst	Stabil?
Insertion Sort	$O(n)$	$O(n^2)$	ja
Selection Sort	$O(n^2)$	$O(n^2)$	nein
<code>std::sort</code>		$O(n \log n)$	nein

10.2 Graphen

Definition: Graph

Ein Graph $G = (V, E)$ besteht aus **Knoten** (Vertices V) und **Kanten** (Edges E), die paarweise Beziehungen darstellen. Eigenschaften: *gewichtet/ungewichtet, gerichtet/ungerichtet, zusammenhängend*, Grad eines Knotens (Anzahl Kanten).

Adjazenzmatrix vs. Adjazenzliste

	Adjazenzmatrix	Adjazenzliste
Struktur	$n \times n$ -Matrix, [i][j]	pro Knoten Liste der Nachbarn
Kante prüfen	$O(1)$	$O(\text{Grad})$
Speicher	$O(V^2)$	$O(V + E)$
gut bei	dichten Graphen	dünnen (sparse) Graphen


```

// Adjazenzmatrix (gewichtet) ueber 2D-Vektor:
std::vector<std::vector<int>> matrix(n, std::vector<int>(n, 0));
matrix[i][j] = gewicht;           // Kante i->j mit Gewicht

// Adjazenzliste (gewichtet): pro Knoten Liste von (Nachbar, Gewicht)
std::vector<std::vector<std::pair<int,int>>> adj(n);
adj[i].push_back({j, gewicht});

```

Dijkstra – kürzester Pfad

Findet die kürzesten Pfade von einem Startknoten zu allen anderen in einem **gewichteten** Graphen. Idee: stets den unbesuchten Knoten mit der kleinsten bekannten Distanz wählen und dessen Nachbarn aktualisieren (*Relaxation*).

Merke · Prüfungstipp

- **Effizienter** als alle Pfade zu berechnen, weil bereits gefundene kürzeste Distanzen wieder verwendet werden (gierige Auswahl) statt jede Kombination zu prüfen.
- Benötigt **nicht-negative** Kantengewichte – bei negativen Gewichten kann ein als „fertig“ markierter Knoten nachträglich noch verbessert werden, was Dijkstra nicht berücksichtigt.

11 Tooling & Testing (Lab 12)

11.1 Unit-Tests mit GoogleTest

Unit-Tests prüfen einzelne Bausteine (z. B. eine Klasse) automatisiert. GoogleTest (gtest) ist ein verbreitetes Framework.

Syntax

```
TEST(TestSuiteName, TestName) { ... Assertions ... }
```

```
#include <gtest/gtest.h>
#include "calculator.hpp"

TEST(CalculatorTest, Addition) {
    Calculator calc;
    EXPECT_EQ(calc.add(2, 3), 5);
    EXPECT_EQ(calc.add(-1, 1), 0);
}

TEST(CalculatorTest, DivisionByZero) {
    Calculator calc;
    EXPECT_THROW(calc.divide(10, 0), std::invalid_argument); // erwartet Exception
}
```

Assertion	prüft
EXPECT_EQ(a, b)	a == b (Test läuft bei Fehler weiter)
EXPECT_TRUE(x) / FALSE	Wahrheitswert
EXPECT_THROW(expr, Typ)	dass <code>expr</code> eine Exception vom Typ wirft
ASSERT_*	wie EXPECT_*, aber bricht den Test bei Fehler ab

Best Practice · Konvention

EXPECT_* meldet einen Fehler, lässt den Test aber weiterlaufen (mehrere Prüfungen pro Test). ASSERT_* bricht sofort ab – sinnvoll, wenn die folgenden Zeilen sonst abstürzen würden (z. B. nach einem Null-Check).

11.2 Build mit CMake

CMake beschreibt den Build plattformunabhängig in CMakeLists.txt. FetchContent lädt Abhängigkeiten (hier gtest) automatisch.

```
cmake_minimum_required(VERSION 3.20)
project(gtest_lab LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

enable_testing()
add_library(calculator src/calculator.cpp)
target_include_directories(calculator PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include)

include(FetchContent)
FetchContent_Declare(googletest
    URL https://github.com/google/googletest/archive/refs/tags/v1.17.0.zip)
FetchContent_MakeAvailable(googletest)

add_executable(calculator_tests tests/calculator_tests.cpp)
target_link_libraries(calculator_tests PRIVATE calculator GTest::gtest_main)

include(GoogleTest)
gtest_discover_tests(calculator_tests)
```

Bauen & Testen typischerweise:

```
cmake -S . -B build      # Build-Verzeichnis konfigurieren
cmake --build build      # kompilieren
ctest --test-dir build   # Tests ausfuehren
```

Merke · Prüfungstipp

Grundbausteine einer CMakeLists.txt: `add_library` / `add_executable` (Ziele), `target_link_libraries` (Abhängigkeiten/Bibliotheken), `target_include_directories` (Header-Pfade). `gtest_discover_tests` registriert die Tests automatisch bei CTest.

12 Coding-Conventions (Kurzreferenz)

Komprimierte Regeln aus `cpp_coding_conventions.md` (an MISRA-C++-Ideen angelehnt). Leitsatz:

Merke · Prüfungstipp

„Code is written for humans first and for the compiler second.“ Lesbarkeit und Sicherheit sind meist wichtiger als eine gesparte Zeile. Ziel: lesbar, wartbar, testbar, explizit, robust.

12.1 Namensgebung

Element	Stil
Variablen / Parameter	lower_snake_case oder camelCase
Funktionen / Methoden	dito, möglichst Verben
Namespaces	lower_snake_case
Konstanten	UPPER_SNAKE_CASE
Klassen / Structs / Enums	PascalCase
Pointer	beginnen mit p_ bzw. ptr_

Wichtig: Egal welcher Stil – **konsistent** bleiben.

12.2 Formatierung

4 Leerzeichen (oder 1 Tab) pro Ebene, Tabs & Spaces **nicht mischen**; eine Anweisung pro Zeile; **immer Klammern** `{}` bei Kontrollstrukturen; Zeilen $\lesssim$ 100 Zeichen; Leerzeile zwischen logischen Blöcken.

```
// Gut                                // Schlecht
if (temperature > MAX_TEMPERATURE) {   if (t > 80) shutdown_heater();
    shutdown_heater();                // unklarer Name, magische Zahl, keine Klammern
}
```

12.3 Dateien & Header

Ein Modul = ein Header (`.hpp`, Schnittstelle) + eine Quelldatei (`.cpp`, Implementierung). Header brauchen **Include-Guards** und enthalten Deklarationen/Typen/Konstanten – keine großen Implementierungen. Jede Quelldatei beginnt mit einem kurzen Datei-Header; öffentliche Funktionen dokumentieren (Parameter, Rückgabe). Erkläre **warum**, nicht jedes triviale *was*.

12.4 Variablen, Typen & Konstanten

- Variablen **sofort initialisieren**, eine Deklaration pro Zeile, Scope minimal.
- Magische Zahlen durch benannte Konstanten ersetzen.
- Passende Typen wählen, keine impliziten Narrowing-Konvertierungen; `bool` für logische Zustände.
- `enum class` statt `enum`; signed/unsigned nicht unbedacht mischen.

12.5 Funktionen & Parameterübergabe

Eine Funktion = eine Verantwortung; kurz halten; Namen beschreiben Verhalten.

Parameter	Übergabe
kleine fundamentale Typen	per Wert (<code>int x</code>)
große Objekte	per Referenz (<code>T& x</code>)
große, nur-lesend	per <code>const</code> -Referenz (<code>const T& x</code>)

12.6 Pointer & Referenzen

Referenzen bevorzugen, wenn `null` kein gültiger Zustand ist; jeden Pointer initialisieren; nie ohne Gültigkeitsprüfung dereferenzieren; `nullptr` statt `NULL`. Globale Variablen vermeiden.

12.7 Kontrollfluss & Ausdrücke

Immer Klammern; flache Verschachtelung; ggf. *early return*. Jedes `switch` braucht `default`; `break` bewusst setzen. Keine überkomplexen Ausdrücke; nicht auf subtile Operator-Präzedenz verlassen – in Zwischenvariablen aufteilen.

12.8 Casts

Keine C-Casts `(int)x`. C++-Casts verwenden und jeden Cast begründen: `static_cast`, `dynamic_cast`, `const_cast`, `reinterpret_cast` (letzteren meiden).

12.9 Klassen & OOP

- Member-Variablen grundsätzlich `private`; öffentliche Methoden = Schnittstelle.
- Eine Klasse = eine klare Verantwortung; Methoden > 5 Zeilen außerhalb der Klasse definieren.
- Alle Member initialisieren, **Member-Initialisierungsliste** bevorzugen; keine komplexe Logik im Konstruktor.
- `const` markieren, wenn der Objektzustand nicht geändert wird (const-correctness).
- Komposition vor Vererbung, außer bei echter „ist-ein“-Beziehung; flache Hierarchien.
- `override` bei überschriebenen Methoden; polymorphe Basisklasse ⇒ **virtueller Destruktor**.
- `static` nur bei wirklich geteiltem Zustand.

12.10 Speicher & Standardbibliothek

Automatische (Stack-)Objekte bevorzugen; `new/delete` nur bei echter dynamischer Lebensdauer – jedes `new` genau ein `delete`, **nie** `new/delete` mit `malloc/free` mischen (besser: Smart Pointer / RAII). `std::string` statt C-Strings, `Standardcontainer` statt manueller Arrays. `using namespace` (v. a. `std`) in größeren Projekten vermeiden.

12.11 Gefährliche Konstrukte

Kein `goto`; nicht auf Undefined Behaviour verlassen; Makros/Präprozessor-Tricks für Konstanten vermeiden (lieber `const/constexpr`).

Best Practice · Konvention

Schnellcheck vor dem Commit: sprechende Namen? keine magischen Zahlen? Klammern überall? Member `private` & initialisiert? `const` wo möglich? kein rohes `new/delete`? `override` gesetzt?